# Strutture di controllo

Salvatore Filippone
salvatore.filippone@uniroma2.it

Federica Montesano
federica.montesano@uniroma2.it



# Indice dei Contenuti

1. Controllo del Flusso (if, if-else, switch)
2. Controllo del Flusso: cicli (for, while)
3. Controllo del Flusso: Istruzioni di Salto
4. Funzioni in C
5. Regole di Scope

# Roadmap

1. Controllo del Flusso (if, if-else, switch)
2. Controllo del Flusso: cicli (for, while)
3. Controllo del Flusso: Istruzioni di Salto
4. Funzioni in C
5. Regole di Scope

# Strutture di controllo: `if` e `if-else`

Praticamente qualunque programma richiede di *alterare* il flusso delle istruzioni in dipendenza dai dati. Primo esempio: `if`

```
// Blocco if semplice
if (condizione) {
    // eseguito se condizione è vera
}
// Blocco if-else
if (condizione) {
    // ramo VERO
} else {
    // ramo FALSO
}
```

```
if (a != 0) {
    c = b / a;
} else {
    printf("Non dividere per zero!\n");
}
```

## Operatore Condizionale (Ternario)

Unico operatore a 3 operandi; scorciatoia per `if-else`:

```
condizione ? espressione1 : espressione2
```

# Cascata di `if-else-if` e `if` Annidati

## Cascata (`if-else-if` ladder)

Valuta le condizioni in sequenza; la prima vera esegue il suo blocco ed esce dalla catena.

```
if (a > 0) {
    printf("Positivo\n");
} else if (a < 0) {
    printf("Negativo\n");
} else {
    printf("Zero\n");
}
```

## `if` Annidati (Nested `if`)

Un blocco `if` posizionato all'interno di un altro `if`.

```
if (a != 0) {
    if (b > 10) {
        c = b / a;
    }
} else {
    fprintf(stderr, "Errore: div per zero!\n");
}
```

# Strutture di controllo: `switch`

```
switch (espressione) {
    case COSTANTE_1:
        // Codice 1
        break;
    case COSTANTE_2:
        // Codice 2
        break;
    default:
        // Codice di default (opzionale)
        break;
}
```

## Regole Fondamentali

- **Tipo dell'espressione**: deve valutare a un tipo intero (`int`, `char`, `enum`)
- **Etichette** `case`: devono essere espressioni costanti e uniche.

## Il meccanismo di Fall-Through

Senza l'istruzione `break`, il flusso di esecuzione continua nei case successivi anche se la loro costante non corrisponde.

# Roadmap

1. Controllo del Flusso (if, if-else, switch)
2. Controllo del Flusso: cicli (for, while)
3. Controllo del Flusso: Istruzioni di Salto
4. Funzioni in C
5. Regole di Scope

# Strutture di controllo: ciclo `for`

Spesso un blocco di istruzioni deve essere *ripetuto* più volte, quando il numero di iterazioni è prevedibile o noto a priori si usa un ciclo `for`:

```
for (INIT; TEST; POST) {
  CODE;
}
```

## Calcolo della potenza $x^n$

```
y=1;
for (i=0; i<n; i++) {
  y = y * x ;
}
```

## Dichiarazione in C99+

A partire da C99 è possibile dichiarare il contatore direttamente nell'inizializzazione: `for (int i = 0; ...)`

# Strutture di controllo: ciclo `while`

Quando il numero di iterazioni non è prevedibile, si usa `while`

```
// L'inizializzazione è esterna
INIT;
while (TEST) {
    CODE;
    AGGIORNAMENTO;
}
```

## Iterazione $3 \cdot n + 1$

```
while (n>1) {
  if (n%2 == 0) {
    n /= 2;
  } else {
    n = 3*n + 1;
  }
}
```

# Strutture di Controllo: Il Ciclo do-while

È l'unico ciclo *exit-controlled*: il codice viene eseguito almeno una volta, poiché il controllo della condizione avviene al termine dell'iterazione.

```
do {
    // Codice (eseguito ALMENO 1 volta)
    AGGIORNAMENTO;
} while (TEST); // <--- Punto e virgola obbligatorio!
```

## Esempio: Validazione Input

```
int voto;

// Chiede l'input finché non è valido
do {
    printf("Inserisci voto (18-30): ");
    scanf("%d", &voto);
} while (voto < 18 || voto > 30);
```

# Cicli Infiniti e Confronto Tra Costrutti

## Cicli Infiniti (Infinite Loops)

Cicli la cui condizione non diventa mai falsa. Usati in server, sistemi embedded o cicli di eventi, solitamente interrotti da `break`.

```c
// Forma con while
while (1) { /* ... */ }

// Forma con for (omissione espressioni)
for (;;) { /* ... */ }
```

## Flessibilità del `for`

Ciascuna delle 3 sezioni del `for` è opzionale:

```c
int i = 0;
for (; i < 5; ) {
    printf("%d\n", i);
    i++;
}
```

| Ciclo | Tipo | Caso d'Uso Principale |
|---|---|---|
| `for` | Entry-Controlled | Numero di iterazioni noto/contatore con range definito. |
| `while` | Entry-Controlled | Iterazioni guidate da condizioni logiche variabili. |
| `do-while` | Exit-Controlled | Garantire almeno una prima esecuzione (menu, input). |

# Roadmap


1. Controllo del Flusso (if, if-else, switch)
2. Controllo del Flusso: cicli (for, while)
3. Controllo del Flusso: Istruzioni di Salto
4. Funzioni in C
5. Regole di Scope

# Istruzioni di Salto (Jump Statements)

## Che cosa sono le Istruzioni di Salto?

Servono ad **alterare il normale flusso di esecuzione** sequenziale del programma, trasferendo immediatamente il controllo del codice a un altro punto.

## `break`

- **Azione**: Interrompe **immediatamente** l'esecuzione del ciclo (`for`, `while`, `do-while`) o dello `switch`.
- **Effetto**: Esce dal ciclo e passa alla prima istruzione successiva al blocco.

## `continue`

- **Azione**: Salta il resto del codice nell'**iterazione corrente**.
- **Effetto**: Non esce dal ciclo, ma passa subito all' *iterazione successiva* (valutando l'incremento/condizione).

## Comportamento nei cicli annidati

Sia `break` che `continue` agiscono esclusivamente sul blocco più interno in cui sono inseriti.

# Istruzioni di Salto: `break` e `continue`

Con l'istruzione `break` si *esce* dal ciclo circostante:

```
while (TEST) {
  if (OTHER TEST) {
    break;
  }
  CODE;
}
OTHER CODE;
```

Con l'istruzione `continue` si passa alla *prossima iterazione* del ciclo

```
for (INIT; TEST; POST) {
  if (OTHER TEST) {
    continue;
  }
  CODE;
}
```

# Roadmap


1. Controllo del Flusso (if, if-else, switch)
2. Controllo del Flusso: cicli (for, while)
3. Controllo del Flusso: Istruzioni di Salto
4. Funzioni in C
5. Regole di Scope

# Le funzioni in C

Una *funzione* racchiude un insieme di istruzioni da eseguire su dati di volta in volta diversi

```
1  TYPE NAME(ARG1, ARG2)
2  {
3    CODE;
4    return VALUE;
5  }
```

## Vantaggi essenziali

- **Modularità**: Scomposizione di problemi complessi in sotto-problemi.
- **Riuso**: Scrivere il codice una volta e invocarlo più volte.
- **Manutenibilità**: Isola i bug e separa l'interfaccia dall'implementazione.

Gli argomenti di una funzione vengono passati *per valore*

# Dichiarazione (Prototipo) vs Definizione

## 1. Dichiarazione (Prototipo)

- Comunica al compilatore la **firma** (nome, tipo di ritorno, parametri).
- Non alloca codice in memoria.
- Termina con il punto e virgola ;.
- Permette di invocare la funzione prima che venga definita.

## 2. Definizione

- Contiene l'**implementazione effettiva** (il corpo con le istruzioni).
- Alloca il codice eseguibile in memoria.
- Deve essere unica nel programma (rule of one definition).

## Regola Fondamentale del Compilatore

Una funzione deve essere dichiarata o definita prima del suo primo utilizzo nel sorgente, altrimenti il compilatore genererà un warning o un errore di simbolo non trovato.

# Passaggio dei Parametri

## Parametri Attuali vs Formali

- **Parametri Attuali (Argomenti)**: I valori o variabili passati al momento della chiamata a funzione.
- **Parametri Formali**: Le variabili locali create nella firma della funzione.

## Passaggio per Valore (Default in C)

- La funzione riceve una **copia** del valore del parametro attuale.
- Le modifiche ai parametri formali **non alterano** le variabili del chiamante.

## Passaggio per Riferimento

- Si passano gli **indirizzi di memoria** usando i puntatori.
- La funzione modifica direttamente il dato originale.

# Esempio Completo di Strutturazione

```c
#include <stdio.h>

// 1. DICHIARAZIONE / PROTOTIPO (Spesso posizionato nei file di header .h)
int mcd(int m, int n);

int main(int argc, char *argv[]) {
    int a = 12, b = 25;

    // 2. CHIAMATA A FUNZIONE
    int c = mcd(a, b);

    printf("MCD tra %d e %d: %d\n", a, b, c);
    return 0; // Termina main e svuota il Call Stack
}

// 3. DEFINIZIONE DELLA FUNZIONE (Implementazione)
int mcd(int m, int n) {
    while (n != 0) {
        int temp = n;
        n = m % n;
        m = temp;
    }
    return m;
```

# La Funzione `main`

## Che cos'è la funzione `main`?

È il **punto di partenza obbligatorio** di ogni programma C. L'esecuzione del codice inizia sempre al suo interno e deve essercene esattamente una in tutto il progetto.

## Le Firme Standard

- `int main(void)`:
  Senza argomenti in ingresso.
- `int main(int argc, char *argv[])`:
  Per ricevere argomenti da riga di comando.

## Il Valore di Ritorno (`int`)

Comunica al sistema operativo com'è andata l'esecuzione:

- `return 0;`: esecuzione completata con successo. Esiste anche la macro EXIT_SUCCESS.
- `return valore;`: segnala un errore (valore $\neq 0$). Esiste anche la macro EXIT_FAILURE

# Roadmap


1. Controllo del Flusso (if, if-else, switch)
2. Controllo del Flusso: cicli (for, while)
3. Controllo del Flusso: Istruzioni di Salto
4. Funzioni in C
5. Regole di Scope

# Regole di Scope (Ambito di Visibilità)

## Che cos'è lo Scope?

Definisce la regione del programma in cui una variabile o identificatore è accessibile.

## I 4 Tipi di Scope in C

1. **Block Scope (Local Scope)**: Dichiarate dentro un blocco `{ }`. Esistono solo fino alla chiusura del blocco.
2. **File Scope (Global Scope)**: Dichiarate fuori da ogni funzione. Visibili dalla dichiarazione a fine file.
3. **Function Scope**: Riservato alle etichette (*labels*) usate con la direttiva `goto`.
4. **Function Prototype Scope**: Nomi dei parametri nei prototipi di funzione.

**Variabili Statiche** (`static`): Mantengono il proprio valore tra chiamate successive della stessa funzione.

# Variable Shadowing (Mascheramento)

## Concetto

Se una variabile in un blocco interno ha lo **stesso nome** di una variabile esterna o globale, la variabile interna **maschera** quella esterna all'interno del proprio blocco.

## Consigli di Stile

- Prediligere variabili locali per evitare effetti collaterali indesiderati.
- Usare le variabili globali con estrema parsimonia.

## Esempio Pratico

```c
#include <stdio.h>

int x = 10; // Global Scope

int main() {
    int x = 20; // Maschera la globale

    {
        int x = 30; // Maschera locale
        printf("%d\n", x); // Stampa 30
    }

    printf("%d\n", x); // Stampa 20
    return 0;
}
```